

Delta Lake MERGE Implementation

Incremental Data Processing · Databricks Free Edition · PySpark

1. Objective

This assignment demonstrates incremental data processing using Delta Lake — a core pattern in modern data engineering. The workflow covers loading a real-world retail dataset, cleaning it, simulating a batch of incoming updates and new records, and applying a single atomic MERGE (upsert) operation. All steps were executed on Databricks Free Edition using PySpark with Delta Lake built-in.

2. Dataset

Source: Superstore Sales Dataset — Kaggle (vivek468/superstore-dataset-final)

Original size: 9,994 rows × 20 columns

Key columns: Row ID, Order ID, Customer ID, Sales, Quantity, Discount, Profit, Region, Category

Unique key used for MERGE: Row_ID (already present in dataset)

3. Steps Performed

Step	Description	Output
1	Loaded Superstore CSV from Databricks Volume into Spark DataFrame	9,994 rows · 20 columns
2	Fixed column names (spaces → underscores); cast Sales, Quantity, Discount using try_cast	Valid schema
3	Wrote cleaned DataFrame as Delta table customer_master	Delta table created
4	Dropped rows with nulls in Sales, Quantity, Customer_ID; removed exact duplicates	8,771 clean rows
5	Simulated incremental data: 10% held-out rows as new records + 5 modified rows as updates	Incremental dataset ready
6	Applied Delta Lake MERGE INTO on Row_ID: matched → update, unmatched → insert	Upsert complete
7	Validated final row count, distinct key count, and duplicate groups	All checks passed
8	Displayed final Delta table ordered by Row_ID	Final output verified

4. MERGE Operation

The MERGE operation is the centerpiece of this assignment. It performs an atomic upsert — updating matched rows and inserting new rows in a single transaction, ensuring ACID compliance:

```
delta_master.alias('t')
  .merge(df_incr.alias('s'), 't.Row_ID = s.Row_ID')
  .whenMatchedUpdate(set={'Sales': 's.Sales', 'Quantity': 's.Quantity', 'Discount':
's.Discount'})
  .whenNotMatchedInsertAll()
  .execute()
```

This eliminated the need for separate UPDATE and INSERT statements. Matched records (by Row_ID) were updated in-place; unmatched records were inserted as new rows.

5. Validation Results

Metric	Value	Status
Final row count	8,771	✓ Pass
Distinct Row_ID count	8,771	✓ Pass
Duplicate Row_ID groups	0	✓ Pass

6. Key Learnings

- **ACID compliance:** Delta Lake MERGE guarantees atomicity — no partial writes or inconsistent states.
- **Column naming:** Delta tables reject spaces/hyphens in column names; renaming is mandatory before writing.
- **Safe casting:** try_cast handles malformed/empty values gracefully, returning NULL instead of failing.
- **Incremental pattern:** Splitting one source dataset into master + incremental simulates real ETL pipelines.
- **Unique key:** A reliable unique key (Row_ID) is critical — MERGE correctness depends entirely on it.